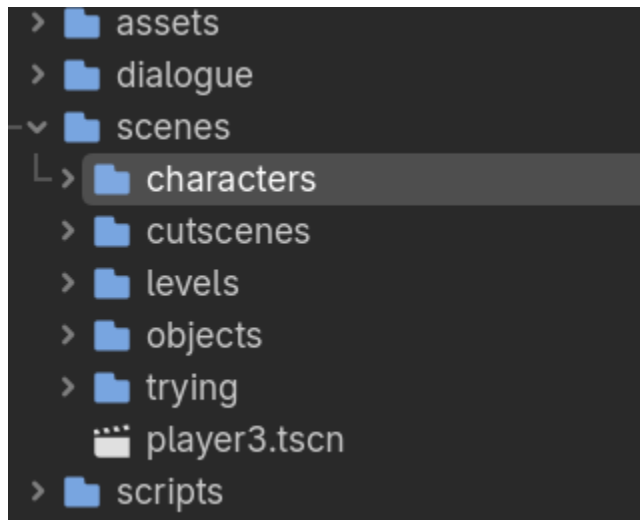


Technical Design Document

So you want to expand on this game.

General File Structure



All files will be in their respective folders.

So all scenes should be under the **scenes** folder and all scripts under the **scripts** folder. My teammate chooses to do this differently and organizes them instead by relevance but for my part of the code things are organized this way.

The **scenes** folder is made up of 5 subfolders. **Characters** contain all of the

characters in the story, which so far is the mermaid, the shrimp and the hoard of goldfish.

Cutscenes contain all of the cutscenes that will be played in order to advance the story.

Levels contain the actual playable levels. **Objects** are parts of the level, so moving platforms, levers, pushable objects, minimaps will all be stored here. **Trying** is the folder of ideas. Anything that you want to try out that either didn't make the cut or was out of scope will end up here.

The only exception to this is the **player3.tscn**. This stays outside for ease of access

The scripts are at this moment not very well organized but they are named mostly after the scenes.

Creating a new level

Firstly, put the level in the correct subfile in the file structure described above. The level switching is done entirely in the **game_manager.gd**.

Spawn points and Exits

The game manager assumes that the level will have **spawn_points** (objects part of the group called **spawn points** whose location will be used to spawn the player at that point) and **exits** (objects part of the group called **exits** that will be used to signal the end of the level and the start of the next). As of this moment, determining which scene the player is `get_tree().current_scene.name` currently in to load behavior that is specific to that scene is done very clunkily as the full function is called at multiple parts of the `game_manager.gd`

All spawn points are of the class **Marker2D**. Theoretically any object with a fixed global position can be used.

All exits emit `signal loadNewArea(current_scene, exit)` which is information about the `current_scene` and which the current exit is. The bare bones infrastructure does exist in `game_manager.gd` to implement multiple exits from a level that can lead to different levels (i.e. branching progression) but at the moment, all exits are simply marked "**main**" and this functionality is mostly ignored. When adding an object to **exits**, ensure they have an **Area2D** node and emit the `loadNewArea` signal.

Level Progression

Be sure to add the preloaded scene to the variable list here

```
var level1_tank1: PackedScene = preload("res://scenes/levels/level1_tank1.tscn")
var level1_sewer: PackedScene = preload("res://scenes/levels/level1_sewer.tscn")
var level1_boat: PackedScene = preload("res://scenes/levels/level_1_boat.tscn")
var end_screen: PackedScene = preload("res://scenes/levels/end_scene.tscn")
```

Then edit the if/elif box here to load it

```
func _switch_level(current_scene) -> void:
    var next_level := _get_new_level(current_scene)
    if next_level == "level1_tank1":
```

```

        get_tree().change_scene_to_packed(level1_tank1)
    elif next_level == "level1_sewer":
        get_tree().change_scene_to_packed(level1_sewer)
    elif next_level == "level1_boat":
        get_tree().change_scene_to_packed(level1_boat)
    elif next_level == "end_scene":
        get_tree().change_scene_to_packed(end_screen)

    await get_tree().process_frame
    await get_tree().process_frame
    var scene := get_tree().current_scene
    current_scene = scene
    if scene == null:
        return

    _setup_level()

```

This area of code can probably be improved to a much smaller function that is called multiple times that refers to a variable that has the same name as the string. I didn't realize how clunky it was until the it was already towards the end and had no desire to mess around with types in Godot. This function is only concerned with loading the new level.

Additionally, set the linear progression of levels under **`_get_new_level`**

```

func _get_new_level(current_level) -> String:
>| #if current_level_index < LEVELS_LIST.size()-1:
>| >| #var level_name = LEVELS_LIST[current_level_index + 1]
>| >| #current_level_index += 1
>| >| #return level_name
>| #else: return LEVELS_LIST[1]
>| if current_level.name == "start_scene":
>| >| return "level1_tank1"
>| elif current_level.name == "level1_tank1":
>| >| return "level1_sewer"
>| elif current_level.name == "level1_sewer":
>| >| return "level1_boat"
>| elif current_level.name == "level1_boat":
>| >| return "end_scene"
>| else:
>| >| return "level1_tank1"

```

This can definitely be abstracted to some kind of list which you can see I did try to do in the comments but it was already at the end when I realized how clunky it was.

If you would like to have non_linear progression (via different exit doors, for example), this will be set here as well. This function is primarily concerned with all the logic on which level will come next

Tutorial / Hint text

A level can also include dialogue tutorials. These are dialogue boxes that will be displayed at the start of the level and can be used to introduce the current level as part of the story or introduce a new mechanic that is valid only for that level. The text to be displayed will be listed under the function **_give_instructions()**.

```
func _give_instructions() -> void:
>I  var dialogue_text: String
>I  if get_tree().current_scene.name == "level1_tank1":
>I  >I  dialogue_text = ""~ start
```

Ensure that the text is using the already declared local variable **dialogue_text** and has **~ start** before any text and has **=> END** before the final closing quotation marks. An example text is included below:

```
""~ start
Welcome to the final tank!
Good luck!
=> END
""
```

If any further customization is desired, please refer to the [documentation](#) of Dialogue Manager 3. This is only the very bare bones implementation. Please be aware that Dialogue Manager 3 does not always give meaningful errors and it can be difficult to figure out which line of code tripped a particular error.

Player spawning

```
func _spawn_player() -> CharacterBody2D:
    spawn_points = get_tree().get_nodes_in_group("spawn_points")
    if spawn_points.is_empty():
        return

    var _player = player_scene.instantiate() as CharacterBody2D
    var camera := Camera2D.new()
    _player.add_child(camera)
    camera.zoom = Vector2(3.0, 3.0)
    if get_tree().current_scene.name == "start_scene":
        camera.offset = Vector2(85,0)

    _player.get_node("fish_bg").camera_node = camera
    if get_tree().current_scene.name == "level1_tank1":
        get_tree().current_scene.player_node = _player
        for child in _player.get_children():
            if is_instance_of(child, Camera2D):
                get_tree().current_scene.camera_node = child

    get_tree().current_scene.add_child(_player)
    _player.global_position = spawn_points[0].global_position

    _player.process_mode = Node.PROCESS_MODE_DISABLED

    _add_minimap(_player)
    return _player
```

This is the main code used to spawn the player and the camera. If your level needs to access the **Player** or the **Camera2D** node, these values will be assigned here. Ensure that the levels declare these values

```
var player_node
var camera_node
```

If enough levels need access to the player or camera, it can be changed so that all values will have the variables **player_node** and **camera_node**. Not all levels have a script at the moment which is why they are manually checked depending on which levels need it.

If a different zoom or offset of the main camera is desired, please add it to the section

```
camera.zoom = Vector2(3.0, 3.0)
if get_tree().current_scene.name == "start_scene":
    camera.offset = Vector2(85,0)
```

The default zoom will be **Vector2(3.0,3.0)**. This can and probably should be extracted to a `top_level` variable for cleaner code

Do not fret about the **process_mode** being disabled. This ensures that on spawning, the player's movements are temporarily disabled for the cutscenes to play. It is resumed on cutscene end

```
func _on_dialogue_ended(resource) -> void:  
    player.process_mode = Node.PROCESS_MODE_PAUSABLE
```

If you would like the player to still have some range of movement while the dialogues are being played, you are very welcome to figure out how to disable only certain movements and not others. They can be found under the official Godot documentation [here](#)

Adding the minimap

The player code calls this function and passes the newly created **_player** instance. There is at this moment also a global variable called **player** that also holds reference to the player instance. This is bad code design because I started with the function passing and then switched to global variables. Feel free to coerce the code to one or the other

```
func _add_minimap(_player) -> void:  
    current_scene = get_tree().current_scene  
    var _minimap = minimap_scene.instantiate()  
    _minimap.name = "mini-map"  
    _minimap.player_node = _player  
  
    var _minimap_sewer = _minimap.find_child("mini_map_sewer", true,  
false)  
  
    current_scene.add_child.call_deferred(_minimap, true)  
    _assign_goal_object.call_deferred(get_tree().current_scene, _minimap)  
  
    if _minimap_sewer:  
        _minimap_sewer.player_node = _player
```

There are a lot of code using **call_deferred**, this is because Godot is horrible with scene instantiation timing and has made my life hell while debugging it. The **_switch_level** code also has a lot of waiting around. This function does NOT use `await get_tree().process_frame` because this then runs the function **_process()** inside the minimaps before the player is instantiated, causing it to crash.

I have no idea if this can be optimized, this was the first thing that worked.

This scene switching is also why the **player** and **camera** should ALWAYS be assigned in the **game_manager.gd**. Spare yourself the suffering I went through.

Minimaps require **goal_object** representing the objective of the level (i.e. a character or an exit) with the exception of the second level. They also have a **goal_marker**, the sprite used to render on the minimap representing the **goal_object**. I understand this is stupid coding, but it didn't really make sense to standardize it when I had only 3 levels

```
func _assign_goal_object(current_level, _minimap) -> void:
>| if current_level.name == "level1_tank1":
>| >| _minimap.goal_object = current_level.get_node("goldfish")
>| >| _minimap.goal_marker.texture = goldfish_texture
>| elif current_level.name == "level1_boat":
>| >| _minimap.goal_marker.texture = shrimp_texture
>| >| _minimap.goal_object = current_level.get_node("shrimp")
```

You can probably disable the minimap too in the **_spawn_player()** code that calls the **_add_minimap()** function. I have no idea if anything is depending on it and I didn't test, but it is possible.

Adding objects to levels

Moving platforms

The easiest thing to add is simply a platform that can be animated. The **AnimationPlayer** needs to be manually added as a child of the object inside the scene. The position animation key frames need to then be manually set.

Levers

A lever can be pushed in order to activate or deactivate an object. I used it to start the moving platforms.

```
func activate_lever():
>I  is_on = true
>I  lever.animation = "activate"
>I  lever.play()
>I  object_to_be_activated.get_node("AnimationPlayer").play("move")

func deactivate_lever():
>I  is_on = false
>I  lever.animation = "deactivate"
>I  lever.play()
>I  object_to_be_activated.get_node("AnimationPlayer").stop()
```

Ensure that the object that should be activated is in the group **activate_able**, has an animation called **move** and a child node called **AnimationPlayer**. Ensure that the RESET track (if exists) puts the platform inside the range of the lever. This took some debugging but manually moving the platform does nothing if the position key frames inside the RESET track isn't changed.

I'm not sure how this would work if there were multiple AnimationPlayers or multiple movement patterns, so feel free to expand upon this.

I made it so the player has to push on it a bit for the lever to start for a fixed amount of time that is randomized when the lever is generated, between 0.5 and 1.5 seconds. This range can be changed. I wanted to vary it simply so there would be some difference for the player.

```

func _ready() -> void:
    time_to_switch = rng.randf_range(0.5,1.5)
    influence_area.monitoring = true

    await get_tree().process_frame
    for obj in influence_area.get_overlapping_bodies():
        if obj.is_in_group("activate_able"):
            object_to_be_activated = obj

```

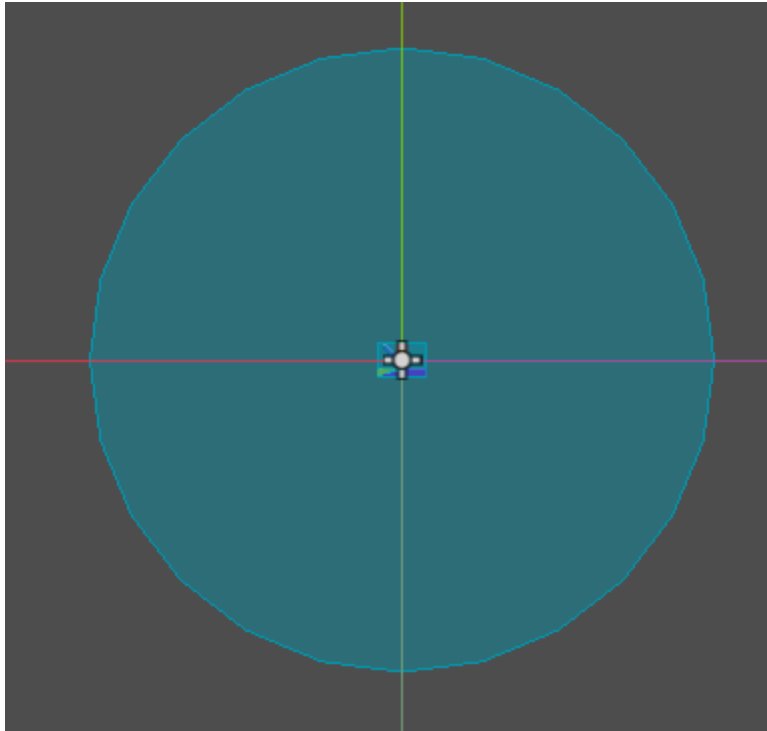
Additionally, aside from the range, the moving platform is set here in the `_ready()` code. Below is the code for the player to push. It simply checks if the player has been constantly pushing on the lever longer than its **time_to_switch** (set in **_ready**). It moves the switch position by playing an animation. If the player stops moving, the timer resets and the player has to push on it more for it to switch

```

func _process(delta: float) -> void:
    for obj in player_move_area.get_overlapping_bodies():
        if obj is CharacterBody2D:
            # now check for direction of travel
            var dir := Input.get_axis("move_left", "move_right")
            if dir > 0 && !is_on: # player moving right and lever is
on the left
                pushing_time += delta
                if pushing_time > time_to_switch:
                    activate_lever()
            elif dir < 0 && is_on:
                pushing_time += delta
                if pushing_time > time_to_switch:
                    deactivate_lever()
            elif dir == 0:
                pushing_time = 0 # reset if not pushing

```

Here is the scene of the lever. The influence area is much larger than the collision area.



Pushable objects

From the perspective of the object, it simply needs to be added to the group **pushable_object** and have a **CollisionShape2D**, the rest is managed by [the player](#)

Pullable objects (object to player)

From the perspective of the object, it simply needs to be added to the group **to_player** and have a **CollisionShape2D**, the rest is managed by [the player](#)

Pullable objects (player to object)

From the perspective of the object, it simply needs to be added to the group **to_object** and have a **CollisionShape2D**, the rest is managed by [the player](#)

Pullable objects (player to object)

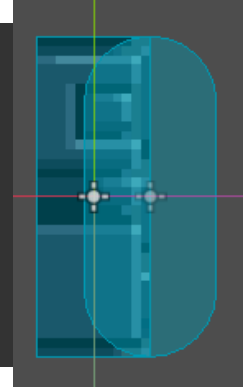
From the perspective of the object, it simply needs to be added to the group **pickup_able** and have a **CollisionShape2D**, the rest is managed by [the player](#)

Climable wall

There is a script called **climable_wall.gd** that simply has an **Area2D** node that sets a boolean **is_climbing** that will be further managed by the Player script

```
func _on_climb_area_body_entered(body: Node2D) -> void:
    if body is CharacterBody2D:
        body.is_climbing = true

func _on_climb_area_body_exited(body: Node2D) -> void:
    if body is CharacterBody2D:
        body.is_climbing = false
```



Brains (Collectibles)

Brain objects **move automatically** and are something the player can collect.

- The movement is set in the code below

```
func _ready():
    start_pos = self.global_position
    start_tween()

func start_tween():
    tween =
get_tree().create_tween().set_process_mode(Tween.TWEEN_PROCESS_PHYSICS)
    tween.bind_node(self)
    tween.set_loops().set_parallel(false)
    tween.tween_property(self, "position", start_pos - DISTANCE, 4.0 / 2)
    tween.tween_callback(Callable(sprite, "set_flip_h").bind(true))
    tween.tween_property(self, "position", start_pos, 4.0 / 2)
    tween.tween_callback(Callable(sprite, "set_flip_h").bind(false))
```

- Additionally, this also flips the sprite. The sprite already faces left which is why it's a bit reversed compared to tutorials
- **DISTANCE** is a Vector2 set to (64,0). This is approximately 4 tilemap tiles to the left
- The 4.0 is the amount of time it takes. This can also be changed

- Please refer to the [documentation](#) for more information on how to modify tweens
- Collectible
 - From the perspective of the collectible, it is only concerned with itself

```
func _on_area_2d_body_entered(body: Node2D) -> void:
    if body is CharacterBody2D:
        brain_collected.emit()
        queue_free()
```

- It emits a signal (that should be connected to from the level) and removes itself from the scene
- For an example, refer to [the tank level](#) on how it can be integrated into a level

The Player

There are 4 player scripts: [player.gd](#), [player_3.gd](#), [player_float.gd](#), [player_wall_climb.gd](#).

The one currently used is **player_3.gd**.

However, I will briefly go over what each of them do and their functionality, in case you would like to steal some functions and add new features to the game

Run, Jump, Dash

The very first player script is called [player.gd](#). This contains the code for various parameters of run, jump and dash

In order for this to work, find the script called **testing_controls_game_manager.tscn** and set this as an Autoload under Project > Project Settings > Globals.

- Ensure that the Name is **GameManager**, the code will not work otherwise. This is because the [player.gd](#) script references this by name

I will go over what this GameManager does. This is not used in the project in anyway and should be removed except for when testing out multiple values of the same controls

There are two classes I declared that is akin to structures used in Java. I just wanted a data structure with pre-defined variable names and data types

```
class TestCase:
    var Function: Callable
    var VarName: String
    var Inputs: Variant # should be type any but it's not there??

class PlayerDefaults:
    var Acceleration: float
    var MaxSpeed: float
    var JumpVelocity: int
    var JumpStyle: String
    var DashDistance: int
    var DashTime: float
```

- Each **TestCase**, generically speaking is a **Function** that sets a particular **VarName** (variable) to a specific value (**Inputs**)
- A **PlayerDefault** is the total list of values determining player characteristics that the player will default to when that variable is not the one currently being tested

The test cases are first generated and then started.

```
func _ready() -> void:
    set_default_variables(DEFAULTS)

    var test_cases: Array[TestCase] = []
    for scenario in SCENARIOS:
        var tests: Array[TestCase] = generate_test_case(scenario,
SCENARIOS[scenario])
        for t in tests:
            test_cases.append(t)
    start_test_case(test_cases)
```

- There are two constants that need to be decided: **SCENARIOS** and **DEFAULTS**
 - **SCENARIOS** is a Dictionary of the test cases in the format "VARIABLE NAME" :
[Array of values that this variable will take]
 - Ensure that double quotation marks are used "" bc Godot is stupid

- **DEFAULTS** are the values that will be set and kept as control between test cases so the only variable changing is the one being set

```
const SCENARIOS = {
  "JUMP_STYLE": [ "hold_jump", "double_jump",
"hold_hover","simple_jump"],
  "ACCELERATION": [5,10,INF],
  "MAX_SPEED": [400,500,8000],
  "JUMP_VELOCITY": [-300,-400,-500],
  "DASH_DISTANCE": [100,200,1000],
  "DASH_TIME": [0.25, 0.8, 1.2]
}

const DEFAULTS = {
  "ACCELERATION": INF,
  "MAX_SPEED": 400,
  "JUMP_VELOCITY": -350,
  "JUMP_STYLE": "simple_jump",
  "DASH_DISTANCE": 100,
  "DASH_TIME": 0.8,
}
```

- Below is the code on generating the test cases

```
func set_var(variable_name: String, val)->void:
  set(variable_name, val)

func generate_test_case(variable_name: String, values: Array) -> Array:
  var test_cases: Array[TestCase] = []
  for v in values:
    var a = TestCase.new()
    a.Function = set_var
    a.VarName = variable_name
    a.Inputs = v
    test_cases.append(a)
  return test_cases
```

- Every test case is basically setting a variable to a certain value. Thus, it has three parts: the actual **Callable** that does the value setting, the **VarName** that is the variable whose value will be changed and **Inputs** which is what this variable will be set to

- This is the code that starts the test cases

```
func _on_timer_timeout():
    get_tree().reload_current_scene()

# should pass a structure of function and input
# so of the format {"function": , "input": }
func start_test_case(fs: Array[TestCase]) -> void:
    for x in fs:
        for child in get_children():
            if child is Timer:
                child.queue_free()

        set_default_variables(DEFAULTS)
        print("CURRENT VAR: " + str(x.VarName))
        print("VALUE: " + str(x.Inputs))
        var callable = x.Function
        callable.call(x.VarName, x.Inputs)

        var timer = Timer.new()
        add_child(timer)
        timer.wait_time = 20
        timer.start()
        timer.timeout.connect(_on_timer_timeout)
        await timer.timeout
```

1. If there is already a Timer, remove it from the scene
2. Set all variables to its defaults
3. Get the function from the test case and call it to set the given variable to the given value
4. Start a timer of 20 seconds
5. On timer timeout, reload the scene and the for loop continues

Yes it's over-engineered. I know. This is the best way I could come up with for automated testing of these values and I didn't expect to not be able to set these values in the GameManager but need to call on them from the Player instead.

Jump Style

These are the various styles of jumping that the player can use and to be tested.

In order to call the different functions, they are stored in a variable

```
var JUMP_STYLE_FUNCTIONS = {  
    "simple_jump" : simple_jump,  
    "hold_hover" : hold_hover,  
    "hold_jump" : hold_jump,  
    "double_jump" : double_jump  
}
```

These functions are then called inside `_physics_process()`

```
var jump_function = JUMP_STYLE_FUNCTIONS[JUMP_STYLE]  
jump_function.call(delta)
```

`JUMP_STYLE` is a variable that is emitted by the GameManager, and is declared in [player.gd](#)

```
@onready var JUMP_STYLE = GameManager.JUMP_STYLE
```

- The most basic jump is the simple jump. Player presses a button, character goes up and then comes back down according to the gravity

```
func simple_jump(delta):  
    if Input.is_action_just_pressed("jump") and is_on_floor():  
        velocity.y = JUMP_VELOCITY
```

- This is the double jump allowing the player to jump up to twice consecutively.
 - There is a variable called **number_already_jump** that logs the number of times the player has already pressed the jump button.
 - The function applies the vertical velocity as long as the player has not pressed jump more than twice.
 - If the player is on the floor, this number resets to 0

```
func double_jump(delta):  
    if number_already_jump < 2 && Input.is_action_just_pressed("jump"):  
        velocity.y += JUMP_VELOCITY  
        number_already_jump += 1
```

```
if is_on_floor_only() && number_already_jump > 0:
    number_already_jump = 0
```

- **hold_hover**: if the player holds down the jump button, they will hover mid air and eventually fall back down
 - **MAX_HOVER_TIME** is set to 1.5 (seconds)
 - Nothing happens when the player pressed jump if they have been hovering for longer than this amount of time
 - **MAX_JUMP_HEIGHT** is the maximum height a player can go up, set to 160
 - If the player's position is above this limit, reset it to this limit and reduce the player's vertical velocity
 - Otherwise, add the vertical velocity
 - The numbers are set to be less than MAX_JUMP_HEIGHT because in Godot y=0 is on top and not on the bottom
 - The **time_pressed_since_pressed_jump** resets when the player releases the jump button

```
func hold_hover(delta):
    if Input.is_action_pressed("jump"):
        if time_passed_since_pressed_jump > MAX_HOVER_TIME:
            pass
        else:
            time_passed_since_pressed_jump += delta
            if position.y <= MAX_JUMP_HEIGHT:
                position.y = MAX_JUMP_HEIGHT
                velocity.y = 0
            else:
                velocity.y = JUMP_VELOCITY

    if Input.is_action_just_released("jump"):
        time_passed_since_pressed_jump = 0;
```

- **hold_jump**: simply hold down the jump button to go higher. There is no upper limit implemented for this so absent a top border it just goes on forever

- **CUT_JUMP_TIME** is how much it goes up the longer the jump button is pressed, it is right now set to 20
- **time_passed_since_pressed_jump** is a global variable simply marking how long since the player pressed the button

```
func hold_jump(delta):
    if Input.is_action_just_released("jump"):
        time_passed_since_pressed_jump = 0;

    if Input.is_action_pressed("jump"):
        time_passed_since_pressed_jump += delta
        velocity.y = JUMP_VELOCITY - time_passed_since_pressed_jump * CUT_JUMP_TIME
```

Jump Velocity

This is how much the player moves vertically by. This value is used in the jump style functions above

Acceleration

This is how fast the player approaches their max speed

```
if direction > 0 && !is_player_dashing:
    time_passed_since_pressed_acceleration += delta
    velocity.x = move_toward(velocity.x, MAX_SPEED, acceleration *
time_passed_since_pressed_acceleration)
elif direction < 0 && !is_player_dashing:
    time_passed_since_pressed_acceleration += delta
    velocity.x = move_toward(velocity.x, -MAX_SPEED, acceleration *
time_passed_since_pressed_acceleration)
```

- **time_passed_since_pressed_acceleration** is the amount of time that the movement button is held down
- I separated the two directions because of the dashing. I wanted the dash to work even if the player was not actively moving in that direction

Max Speed

This is the fastest the player can go. Used in the movement above

Dashing

The code is a bit convoluted so let's walk through it

- On the player pressing "dash", a timer with the value of **DASH_TIME** is started and added to the scene.
- When this timer runs out, the player is no longer dashing

```
var direction = Input.get_axis("move_left", "move_right")
if Input.is_action_pressed("dash"):
    is_player_dashing = true
    var dash_timer = Timer.new()
    add_child(dash_timer)
    dash_timer.wait_time = DASH_TIME
    dash_timer.start()
    await dash_timer.timeout
    is_player_dashing = false
```

Checking which way the sprite is facing

```
if direction > 0:
    animated_sprite.flip_h = false
    is_flip_h = false
elif direction < 0:
    animated_sprite.flip_h = true
    is_flip_h = true
```

There are a couple direction checks

- Player is moving right and is not dashing → accelerate player to max speed in the positive direction (to the right)
- Player is moving left and is not dashing → accelerate player to max speed in the negative direction (to the left)
- Player is moving and is dashing → move the player per frame by the dash speed (calculated by **DASH_DISTANCE/DASH_TIME**)

- Player is not moving and is dashing → get which way the player is facing from its sprite direction and apply the dash in that direction

```

if direction > 0 && !is_player_dashing:
    time_passed_since_pressed_acceleration += delta
    velocity.x = move_toward(velocity.x, MAX_SPEED, acceleration *
time_passed_since_pressed_acceleration)
elif direction < 0 && !is_player_dashing:
    time_passed_since_pressed_acceleration += delta
    velocity.x = move_toward(velocity.x, -MAX_SPEED, acceleration *
time_passed_since_pressed_acceleration)
elif (direction > 0 && is_player_dashing) || (direction < 0 && is_player_dashing) :
    velocity.x = direction * (DASH_DISTANCE/DASH_TIME)
elif (direction == 0 && is_player_dashing):
    var dash_sign = -1 if is_flip_h else 1
    velocity.x = dash_sign * (DASH_DISTANCE/DASH_TIME)
else:
    velocity.x = move_toward(velocity.x, 0, SPEED)
if Input.is_action_just_released("move_left") || Input.is_action_just_released("move_right"):
    time_passed_since_pressed_acceleration = 0

```

Wall Climbing

This is the code inside **player_wall_climb.gd**

The code was shamelessly copied from Mostly Mad Productions (2025). Please watch that video for a better understanding of how the different variables are tied together

Floating (limiting jump)

This is the code inside **player_float.gd**

```

func _physics_process(delta: float) -> void:
    if not is_on_floor() || is_on_wall():
        velocity += get_gravity() * delta
    player_jump(delta)
    var direction := Input.get_axis("move_left", "move_right")
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)
    move_and_slide()

```

```

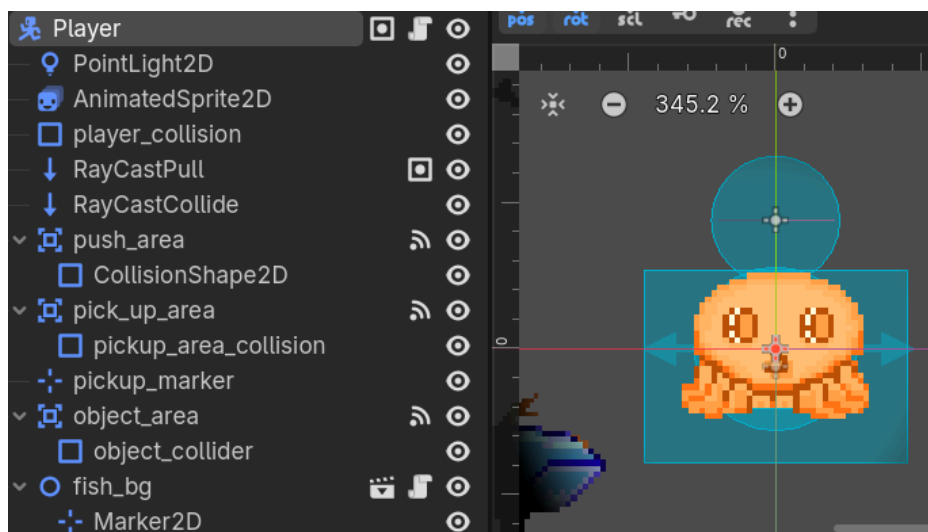
func player_jump(delta: float) -> void:
    if Input.is_action_just_pressed("jump") and (is_on_floor() or
is_on_wall()) and currentTimeToJump >= maxTimeToJump :
        velocity.y = JUMP_VELOCITY
        currentTimeToJump = 0
    if currentTimeToJump < maxTimeToJump:
        currentTimeToJump += delta * jumpReloadRate
    else:
        currentTimeToJump = maxTimeToJump

```

- **maxTimeToJump** is the timer value representing how long the player needs to cool off before they can jump again
- **currentTimeToJump** is the current amount of time left before the player can jump again
 - On player jump, this is set to 0
 - For every delta, this value is incremented by a rate of **delta * jumpReloadRate**
 - **jumpReloadRate** is how fast this bar reloads and is set to 100

Push, Pull, Pick Up

Here are all the components set on the player



Pushing an object

```
func _on_push_area_body_entered(body: Node2D) -> void:
    if body.is_in_group("pushable_object"):
        body.collision_layer = 1
        body.collision_mask = 1

func _on_push_area_body_exited(body: Node2D) -> void:
    if body.is_in_group("pushable_object"):
        body.collision_layer = 2
        body.collision_mask = 2
```

I followed Gwizz (2024)'s tutorial on making an object pushable.

- Set an **Area2D** node on the player called the **push_area**
- If an object part of the group **pushable_object** enters, put it on the same **collision_mask** as the player (so the player's movement affects it)
- When the object exits this area, change the **collision_mask** to a different value so the player's movements no longer affect it

Picking up an object

```
func _on_pick_up_area_body_entered(body: Node2D) -> void:
    if body.is_in_group("pickup_able") && Input.is_action_pressed("pick_up"):
        body.position = pickup_marker.global_position
        pickedObject = body
        body.collision_mask = 2
        body.collision_layer = 2

func _on_pick_up_area_body_exited(body: Node2D) -> void:
    if body.is_in_group("pickup_able") &&
!Input.is_action_pressed("pick_up"):
        body.position = body.global_position
        body.apply_central_force(Vector2(0, GRAVITY_WATER))
        pickedObject = null
        body.collision_mask = 3
        body.collision_layer = 3
```

I loosely followed DevDrache (2024)'s tutorial on how to pick up and move objects. I did not replicate the node as they showed in the video but used the same concept of a **Marker2D** node marking where the picked up object will be held (in my case, above the octopus)

- Set an **Area2D** node on the player called **pick_up_area**
- If an object is part of the group **pickup_able** and the player is pressing the **Pick Up** button, assign it to the variable **pickedObject**
 - This **pickedObject** has the same position as the **Marker2D** node called **pickup_marker** in **_process()**

```
# picking up object
if pickedObject:
    pickedObject.position = pickup_marker.global_position
```

- Assign it on the moment of pickup the same position as this **pickup_marker**
- Set the object's **collision_mask** to 2 so it doesn't trigger the player's collider

On release, it's done a bit differently

- Reset **pickedObject** to null

```
if Input.is_action_just_released("pick_up"):
    pickedObject = null
```

- Assigns the position of the object to the player's current position
- Apply the force of gravity so it falls to the ground
- Sets the object's **collision_mask** to both 1 and 2 so the player still runs into it if they don't press the **Pick Up** button

Additionally, I also have the player drop the **pickedObject** if a **TileMapLayer** is inside the **object_area Area2D** node.

```
func _on_object_area_body_entered(body: Node2D) -> void:
    if body is TileMapLayer && pickedObject:
        pickedObject = null
```

Pullable objects

```
if Input.is_action_pressed("pull"):
    var space_state = get_world_2d().direct_space_state
    mouse_position = get_global_mouse_position()

    var origin = global_position # current position of player
    var pull_dir = (mouse_position - origin).normalized()
    var end = origin + pull_dir * PULL_LENGTH

    var query = PhysicsRayQueryParameters2D.create(origin, end)
    # query.collide_with_areas = true

    var result = space_state.intersect_ray(query)

    if result.has("collider"):
        # move object towards player
        if result.collider.is_in_group("to_player"):
            result.collider.apply_central_impulse(-pull_dir * 2 * origin.distance_to(end))
        # move player towards object
        if result.collider.is_in_group("to_object"):
            velocity = pull_dir * 2 * origin.distance_to(end)
```

This code does the following:

- Gets the space_state
- Gets the global mouse position and the player's position
- Gets the direction from the player to the mouse
- Gets the maximum end of this ray (**PULL_LENGTH** is a constant set to 40)
- See if the ray intersects with any objects
 - If object is in the group **to_player** → apply force to push object towards the player
 - If object is in the group **to_object** → apply velocity to the player to move them towards to the object

Crouch, Move in Sewer, Jump and Run

This is the same script as the section above, [player3.gd](#). The functionalities are just not used so this section contains most of the code that actually affects the player

```
func _ready() -> void:
    r = collision_shape.shape.get_radius()
    current_scene = get_tree().current_scene.name
    if "sewer" in current_scene:
        point_light.visible = true
        fish_bg.visible = false
    else:
        point_light.visible = false
        fish_bg.visible = true
```

- If the player is inside the sewer, turn on the point light (so the player can illuminate the area around them) and turn off the fish background (because there are no fish in the sewer)
- If the player is not inside the sewer, turn off the point light (so the player doesn't have a weird light hovering around them) and turn on the fish background (so it looks more like an aquarium)
- Get the current radius of the collision shape so this can be reduced for **crouch**

```
func _physics_process(delta: float) -> void:
    # in sewer scenes remove gravity
    if "sewer" in current_scene:
        var y_dir := Input.get_axis("move_up", "move_down")
        if y_dir:
            velocity.y = y_dir * SPEED
        else:
            velocity.y = move_toward(velocity.y, 0, SPEED)
    else:
        # Add the gravity.
        if not is_on_floor():
            velocity.y += GRAVITY_WATER * delta

    # Handle jump.
    if Input.is_action_just_pressed("jump") and is_on_floor():
        velocity.y = JUMP_VELOCITY
```

```

var direction := Input.get_axis("move_left", "move_right")
if direction:
    velocity.x = direction * SPEED
    if direction > 0:
        sprite.flip_h = false
    if direction < 0:
        sprite.flip_h = true
else:
    velocity.x = move_toward(velocity.x, 0, SPEED)
# crouch
if Input.is_action_just_pressed("crouch") and is_on_floor():
    collision_shape.shape.set_radius(r/2)

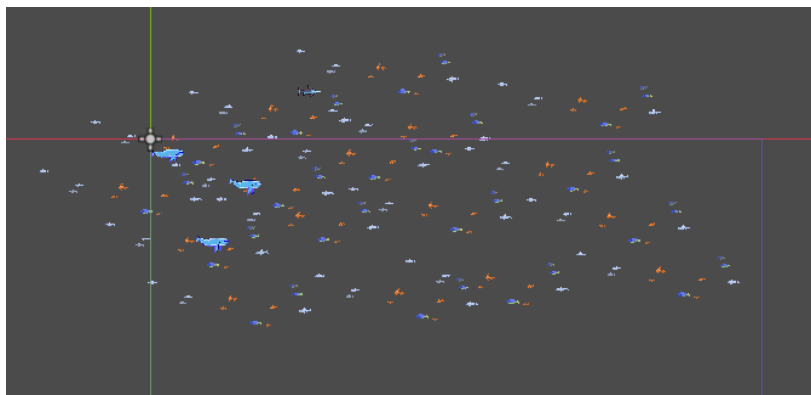
if Input.is_action_just_released("crouch") and is_on_floor():
    collision_shape.shape.set_radius(collision_shape.shape.get_radius()*2)

```

- If the player is currently in the sewer, set the movement in the y direction similar to the x
- Otherwise, add the gravity whenever the player is not currently on the floor
- If the player presses jump, add that vertical velocity
- If the player is moving left or right, change their horizontal velocity and flip the sprite accordingly
- If the player presses **crouch**, reduce the collision_shape's radius to ½
- If the player releases **crouch**, change the collision_shape's radius to it's original value

Fishes in the background

I created a scene file called **fish_bg.tscn** that is linked to the code **fish_bg.gd**



This is what the scene looks like. This makes it so there will be fishes swimming around the octopus. Since they are tagged onto the

octopus, they will move along with it as the octopus moves.

Firstly, it is assumed that the whales and fishes are under a **Node** element called **whales** and **fish**, respectively. The first item in the list of **fish** is one that is on the furthest left. The last item is on the furthest right compared to the others.

They also have different movement speeds, called **SPEED_FISH** and **SPEED_WHALE**

```
@onready var whales: Node2D = $whales
@onready var fish: Node = $fish

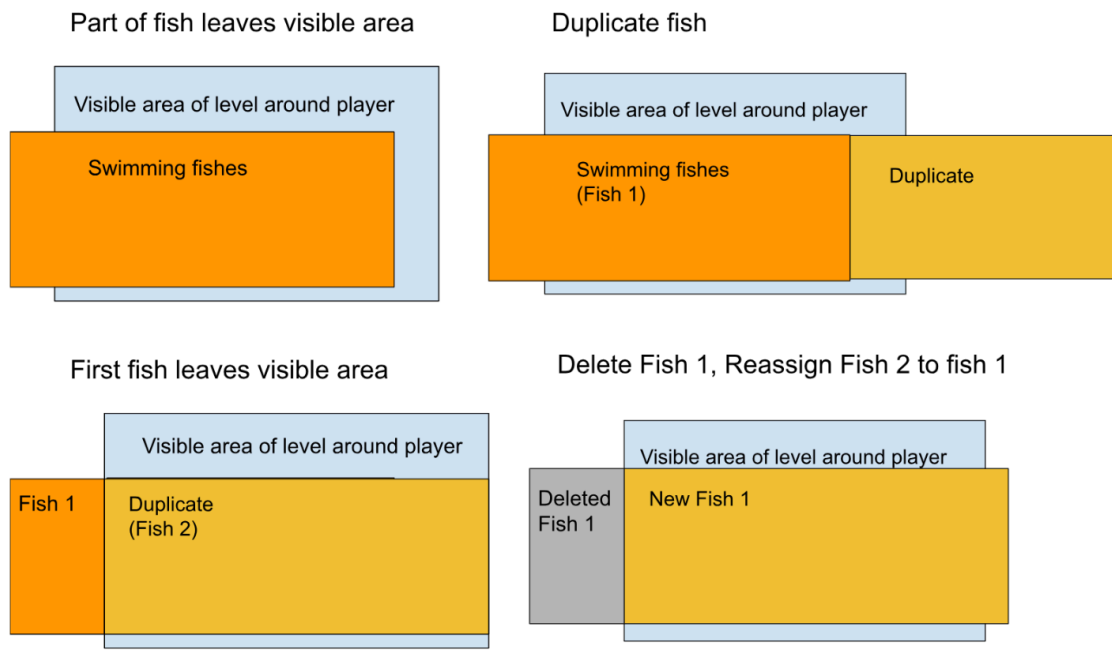
@export var SPEED_FISH: float = 50
@export var SPEED_WHALE: float = 35
```

The existing fish and whales are moved across the screen in **_process()**

```
for f in fish.get_children():
    f.position.x -= SPEED_FISH * delta
if new_fish:
    for f in new_fish.get_children():
        f.position.x -= SPEED_FISH * delta

for whale in whales.get_children():
    whale.position.x += SPEED_WHALE * delta
```

Now onto the spawning logic. The basic premise of the logic is described below



In order to get information about the current position of the herd of fish, and the viewport, some variables are needed

```
func _ready() -> void:
    viewport = get_viewport()
```

```
func _process(delta: float) -> void:
    var center = camera_node.get_screen_center_position()
    var view_size: Vector2i = ((viewport.size as Vector2) /
camera_node.zoom as Vector2i)
    var first_rect: Rect2 = fish.get_children()[0].get_rect()
    var last_rect: Rect2 = fish.get_children()[-1].get_rect()

    var top_left: Vector2 =
fish.get_children()[0].to_global(first_rect.position)
    var bottom_right: Vector2 =
fish.get_children()[-1].to_global(last_rect.end)
    var left_x = center.x - view_size.x/2
    var right_x = center.x + view_size.x/2
    var top_y = center.y - view_size.y/2
    var bottom_y = center.y + view_size.y/2
```

Here is everything the code calculates:

- The center of the screen
- The visible rectangle of the screen
- The bounding rectangle of the fish furthest left (**first_rect**) and furthest right (**last_rect**)
- The top left and bottom right corner. The y doesn't matter here I only needed the x coordinates since the fish will be moving right to left
- Bounding boxes of the visible area (**left_x, right_x, top_y, bottom_y**)

In order to vary where the fish are displayed on scene, I randomize their z-index and make sure everything in my scene has z-indexes between 1 and 4

```
func randomize_z(parent: Node2D):  
    for item in parent.get_children():  
        item.z_index = randf_range(0,4)
```

This is done in **_ready()** for the first school of fish and whales that are spawned

```
rng.randomize()  
  
randomize_z(whales)  
randomize_z(fish)
```

And then the code to spawn the new fish

```
if top_left.x < left_x and !next_fish_here:  
    next_fish_here = true  
    new_fish = fish.duplicate()  
    new_fish.name = "new_fish"  
    randomize_z(new_fish)  
    add_child(new_fish)  
    new_fish.global_position = Vector2(bottom_right.x, top_left.y)  
if bottom_right.x < left_x and new_fish != null:  
    old_fish = fish  
    fish = new_fish  
    new_fish = null  
    next_fish_here = false  
    old_fish.queue_free()
```

```
fish.name = "fish"
```

It checks if the left-most fish has gone over the left side of the visible area and a new fish has not been spawned

- **next_fish_here** set to true so no further fish are spawned
- All the fish (listed under **fish**, a **Node2D**) is duplicated, renamed, have their z randomized and added to the scene
- They are appended to the end of existing school of fish

If the existing fish crosses the left side of the visible area (meaning it has left the visible area), and a new_fish is already here:

- Does a reshuffling, assign current **fish** to **old_fish**, **new_fish** to **fish** and **new_fish** is null
- Removes the **old_fish**
- Renames the **fish** so the code still runs

I do this because I cannot delete **fish** immediately since it would cause an error. It was easier to just create an extra variable that will be deleted from scene and just shuffle everything over by one

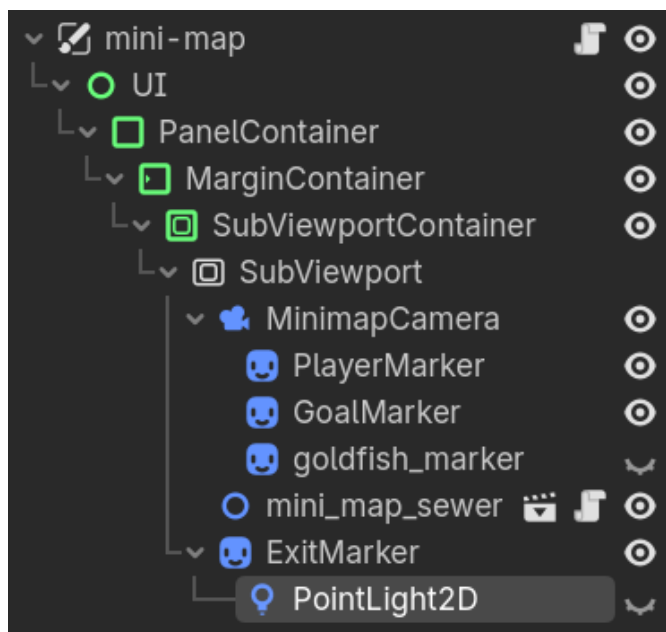
Below is the code for the whales

```
for whale in whales.get_children():
    if whale.global_position.x > right_x:
        if whales.get_child_count() < MAX_WHALES:
            var new_whale = whale.duplicate()
            new_whale.z_index = randf_range(0,4)
            whales.add_child(new_whale)
            new_whale.global_position = Vector2(
                randf_range(left_x-520, left_x-80),
                randf_range(top_y, bottom_y)
            )
            whale.queue_free()
        continue
```

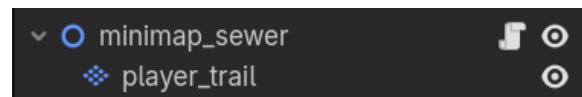
If the whale (travelling left to right) goes off screen and if there are not more than **MAX_WHALES** currently on screen:

- The whale is duplicated, its z-index randomized and added to the scene
- The new global position is set to be between 80 and 520 pixels to the left side of the visible area within the bounds of the scene
- The original whale is removed
- The loop check continues

Minimap



This is the structure of **mini_map.tscn**, **mini_map_sewer.tscn** looks like this,



with **player_trail** being the tilemap that will be painted on in order to show the position of the player

I will first go over the code to paint the position of the player on a tilemap

```
func _physics_process(delta: float) -> void:
    var player_position = (player_node.global_position + Vector2(650,800)) *
    player_trail.get_global_transform()
    var local_pos = player_trail.to_local(player_position)
    var pos_in_tilemap =
    player_trail.local_to_map(player_trail.to_local(player_position))
    player_trail.set_cell(pos_in_tilemap, 0, Vector2(15,1), 0)
```

The **player_position** is first calculated by getting the player's position in the original tilemap and transform it to what it would look like on the tilemap inside the subviewport.

Vector2(650,800) is a value I got via trial and error until the player's movement fit inside the subviewport

The player's position is transformed to a local position, then a tilemap coordinate and then filled in with white

Now comes the code to manage the minimaps with other layers, since there is only one shared **CanvasLayer** across all levels

```
func _ready() -> void:
    # set up scene
    if get_tree().current_scene.name == "level1_sewer":
        is_scene_sewer = true
        minimap_camera.zoom = Vector2(0.2,0.2)
        goal_marker.visible = false
        player_marker.scale = Vector2(0.85,0.85)
        exit_marker.visible = true
        exit_marker.scale = Vector2(1.5,1.5)
        minimap_sewer.visible = true
        goal_object = get_tree().current_scene.get_node("enter_sewer")
    elif get_tree().current_scene.name == "start_scene":
        self.visible = false
    elif get_tree().current_scene.name == "end_scene":
        self.visible = false
    else:
        is_scene_sewer = false
        minimap_camera.zoom = Vector2(0.6,0.6)
        player_marker.scale = Vector2(1.0,1.0)
        exit_marker.visible = false
        minimap_sewer.visible = false

        for tilemap in get_tree().current_scene.get_node("Tilemaps").get_children(): #
ensure all levels with this little guy has a node called TileMaps
            minimap_tilemap = tilemap.duplicate()
            setup_minimap(minimap_tilemap)
            if tilemap.name == "background_layer": # layer with the widest reach
                var used_rect: Rect2i = tilemap.get_used_rect()
                set_minimap_limits(used_rect)
```

This code does the following:

- If the current scene is the sewer
 - Scale the camera and player so it's more zoomed out
 - Disable the **goal_marker**, set the **exit_marker** to be visible
 - This probably should have been made into one single **goal_marker** to make it less confusing but alas
 - Set the **minimap_sewer** to be visible
 - Set the **goal_object** to the exit of the sewer (called **enter_sewer**)
- If the current scene is the start or the end scene, disable the minimap from view
- If the current scene is one of the tank scenes:
 - Reset the scale of the camera and the player marker
 - Make the **exit_marker** invisible
 - Set the **minimap_sewer** to be invisible
 - Copy the tilemaps onto the subviewport

```
func setup_minimap(minimap_tilemap) -> void:
    subviewport.add_child(minimap_tilemap)

func set_minimap_limits(used_rect: Rect2i) -> void:
    minimap_camera.limit_left = used_rect.position.x * 16
    minimap_camera.limit_top = used_rect.position.y * 16
    minimap_camera.limit_right = (used_rect.position.x +
    used_rect.size.x) * 16
    minimap_camera.limit_bottom = (used_rect.position.y +
    used_rect.size.y) * 16
```

setup_minimap simply adds the copied tilemap into the subviewport, while **set_minimap_limits** gets the visible limits of the largest tilemap (the background layer) and ensures the camera doesn't go beyond it

There are two things that need to be rendered on the minimap: the player and the goal object.

In the **minimap_sewer**, the map is painted but we still need to render the little marker denoting the player is there.

It uses a similar code to the **minimap_sewer** to get the position inside the subviewport. For non sewer scenes, the player_node's position is just the **global_position**. Now that I look closer it seems a bit redundant but it works so I'm not going to touch it.

The camera follows a little bit behind the player and the **player_marker** is set to where the player is

```
if player_node:
    var player_position
    if is_scene_sewer:
        player_position = (player_node.global_position + Vector2(650,800)) *
minimap_sewer.get_node("player_trail").get_global_transform()
    else:
        player_position = player_node.global_position - Vector2(0,0)

    minimap_camera.global_position = lerp(minimap_camera.global_position,
player_position, 0.2)
    player_marker.global_position = player_position
```

Now onto rendering the goal.

The main thing I wanted was that if the object is beyond the visible area of the map, it would just render as on the edge of the map and would move away as the player approached it. Thus, I needed to get the boundaries of the area visible on the minimap

```
if goal_object:
    var center = minimap_camera.get_screen_center_position()
    var view_size: Vector2i = ((subviewport.size as Vector2) / minimap_camera.zoom as
Vector2i)
    var pos = goal_object.global_position
    if is_scene_sewer:
        pos.x = clamp(pos.x, center.x - view_size.x/2+OFFSET_EXIT, center.x +
view_size.x/2 - OFFSET_EXIT)
        pos.y = clamp(pos.y, center.y - view_size.y/2+OFFSET_EXIT, center.y +
view_size.y/2 - OFFSET_EXIT)
        #exit_marker.global_position = pos
        exit_marker.global_position = (pos + Vector2(1400,800)) *
minimap_sewer.get_node("player_trail").get_global_transform()
    else:
```

```

        pos.x = clamp(pos.x, center.x - view_size.x/2 +OFFSET_GOAL, center.x +
view_size.x/2 - OFFSET_GOAL)
        pos.y = clamp(pos.y, center.y - view_size.y/2+OFFSET_GOAL, center.y +
view_size.y/2 - OFFSET_GOAL)
        goal_marker.global_position = pos

```

If it's the sewer, the position of the marker should be offset by **OFFSET_EXIT** which is simply how far away from the edge it needs to be so the entire marker still renders

Similar case for regular levels

I do think this makes it a little buggy since sometimes the markers don't move as intended, especially for the sewer level. Feel free to mess around with it though

The Tank Level

Collecting the objects

First, the signals that the collectibles (the brains) will emit are connected to the level in `_ready()`

```

for b in brains.get_children():
    b.brain_collected.connect(_on_collected)

```

```

func _on_collected() -> void:
    num_collected += 1
    inventory.text = str(num_collected)

```

That is all the code needed to track how many the player has collected

Displaying number of collected objects

In the scene called **level1_tank1** there is a **CanvasLayer** with an **HBoxContainer**. This has two items: a **TextureRect** displaying the icon of the brains (the collectible for this level) and a **Inventory** displaying the number of collectibles the player currently has

The inventory text simply displays the internal variable **num_collected** as a string.

Attempting exit of the level

There is a variable called **REQUIRED** setting the number of brains needed. This is not linked to the tutorial text in the GameManager or in the error message when the player attempts to exit the level. This should definitely be changed to make it easier to update this value

If there are enough values collected, the signal **enoughCollected** is emitted which lets the exit know that the player is allowed to exit

If not, a shake is applied

Shaking the screen as visual feedback

The code for the shake is in three parts. There is a RANDOM_SHAKE_STRENGTH set to fixed constant of 30. This is the most violently the camera can be moved.

The actual number the camera moves by is set in **get_random_offset()**

```
func apply_shake() -> void:
    shake_strength = RANDOM_SHAKE_STRENGTH

func get_random_offset() -> Vector2:
    return Vector2(
        rand.randf_range(-shake_strength, shake_strength),
        rand.randf_range(-shake_strength, shake_strength)
    )
```

These values are applied in **_process()**

```
func _process(delta: float) -> void:
    shake_strength = lerp(shake_strength, 0.0, SHAKE_DECAY_RATE * delta)
    camera_node.offset = get_random_offset()
```

In order to have **shake_strength** not be zero and for the shake to take effect,

apply_shake() has to be called. This is done from inside the **_on_exit_attempt()** function

```
func _on_exit_attempt():
    if exit_lock == true:
        return
    else:
```

```

        exit_lock = true
        inventory.add_theme_color_override("font_color",
Color("#54221f"))
        apply_shake()
        player_node.process_mode = Node.PROCESS_MODE_DISABLED
        var reminder_text = "~ start

Oops
Remeber that you need [b]SEVEN[/b] to leave
See how many you have already collected in the top right corner"
        var dialogue =
DialogueManager.create_resource_from_text(reminder_text)
        DialogueManager.show_dialogue_balloon(dialogue, "start")

func _on_player_leave() -> void:
    await get_tree().create_timer(3.0).timeout
    exit_lock = false
    inventory.add_theme_color_override("font_color", Color("#ffffff"))

```

Locking the exit

Since the player is not able to move when inside the **Area2D** marking the exit, if the exit is not stopped, the failure text will keep on retriggering.

This is mainly done with the boolean **exit_lock**.

- Starts off as **false**
- Switches to **true** when the player is attempting to exit. If the player is inside the exit area and **exit_lock** is **true**, nothing will trigger
- In **_on_player_leave()**, wait 3.0 seconds and reset the **exit_lock** to **false**. Player can now attempt to exit again

Visual feedback of inventory text

Additionally, the text is changed to red to indicate failure if the player does not have enough brains collected. This is reset to white when the player leaves the exit area

Exiting level

Tangential to the section above, the one managing the state of the exit is the actual door/character itself. This could probably be changed but I wanted to keep it consistent (the exit manages its own state instead of having it managed by the level's script)

```
func _on_exit_area_body_entered(body: Node2D) -> void:
    if body is CharacterBody2D && is_enough_collected:
        loadNewArea.emit(currentScene, "main")
    else:
        playerArrived.emit()

func _on_exit_area_body_exited(body: Node2D) -> void:
    await get_tree().create_timer(3.0).timeout
    reactivateDoor.emit()
```

It has three signals it emits and the level deals with it accordingly:

- **loadNewArea**: this is the standard exit signal, described in the section about [exits](#)
- **playerArrived**: this is the signal emitted when the player has entered the exit area but does not meet the exit condition
- **reactivateDoor**: this signal is only emitted after the player has left the exit area to prevent playerArrived from retriggering and trapping the player near the exit with no way out. This is done after a timer so it's not accidentally triggered

Additionally, the enoughCollected signal from the current scene is connected which sets the boolean variable (is the exit condition met). This boolean could also be set from the level itself, both are pretty much equivalent

```
func _on_enough_collected():
    is_enough_collected = true
```

Lever tutorial

In order to have the tutorial text be displayed when the player nears a lever (the first one), an **Area2D** node is added to the first lever

There are three main places that deal with this function

This section connecting the signals in `_ready()`

```
DialogueManager.dialogue_ended.connect(_on_dialogue_ended)
lever_tutorial.body_entered.connect(_on_lever_tutorial_body_entered)
```

This section that actually deals with the tutorials of the lever.

I have two checks that are at the moment synonymous but could potentially change in the future if new characters are added:

- If the player is a `CharacterBody2D`
- If the player is part of a group called **player** (that currently has exactly 1 member)

The player's process mode is disabled, similar to the [tutorial text](#) above.

In order to make it fire only once, I disconnect the signal after it has fired

```
func _on_lever_tutorial_body_entered(body: Node2D) -> void:
    if not body.is_in_group("player"):
        return
    if body is CharacterBody2D:
        player_node.process_mode = Node.PROCESS_MODE_DISABLED
        var tutorial_text = ""~ start
        Levers need to be pushed for the moving platforms to activate
        Keep moving in the direction you want to push it in.
        Levers can sometimes be heavy so make sure you keep on pushing. Letting go resets your progress""
        var dialogue = DialogueManager.create_resource_from_text(tutorial_text)
        DialogueManager.show_dialogue_balloon(dialogue, "start")
        lever_tutorial.body_entered.disconnect(_on_lever_tutorial_body_entered)
```

Similar to the [tutorial text](#) above, the player's process mode is reset here

```
func _on_dialogue_ended(resource) -> void:
    player_node.process_mode = Node.PROCESS_MODE_PAUSABLE
```

Sewer level

This is the level with the least amount of technical design behind it. Most of the code setting the new controls are inside the [player code](#) itself. The only thing that is different is the minimap for the sewer

Boat Level

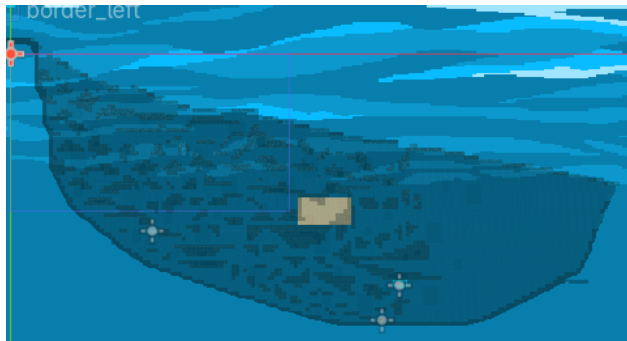
The main code for this level was the one used to spawn the shrimp.

Now I needed to figure out the spawning logic, which included

- Determining the final area (the place that once the shrimp reaches, it will no longer move and wait for the player to come)
- Determining the possible locations where the shrimp can spawn, which should
 - Not be outside the ship
 - Not be behind a filled area of the tilemap
 - Lead the player towards the final area
- Finding an algorithm for the shrimp to lead the player toward the final area

Determining the final area

I created simple tilemap layer that had a filled in rectangle where the final area should be. I then got the coordinates of this rectangle and then removed it from view from inside `_ready()`



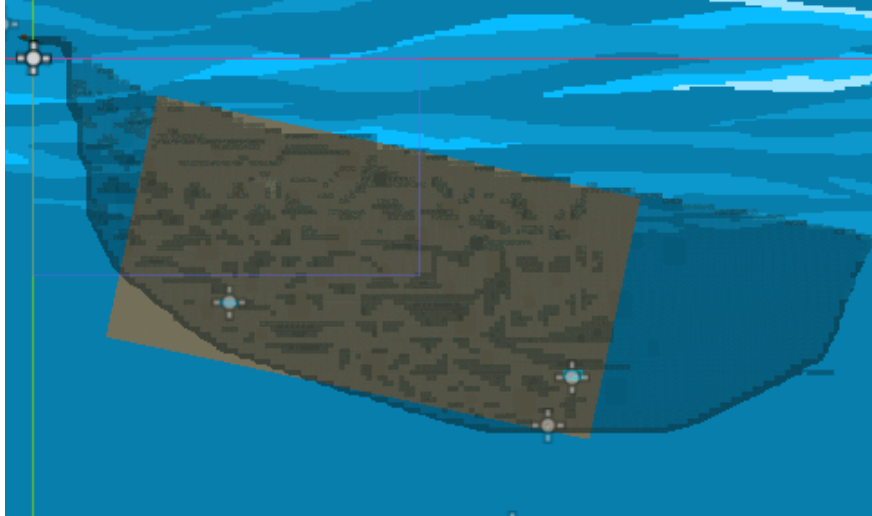
```
final_area = final_area_tilemap.get_used_rect()
final_area_tilemap.visible = false
```

Determining the possible locations where the shrimp can spawn

This one was a bit trickier and involved the three aspects mentioned above

Not be outside the ship

I created a tilemap layer with a filled rectangle and rotated it so that it included most of the ship



I then calculated an ellipse from this rectangle (since the ship is rounded and having the area be an ellipse would reduce the space where the shrimp can spawn that is technically bounded by a rectangle but still outside the physical boundaries of the ship).

This is done by firstly getting the length, width and center of this ellipse, which I do in `_ready()`

```
theta_ellipse = boundary_tilemap.rotation
boundary_tilemap.rotate(-theta_ellipse)
var used_rect = boundary_tilemap.get_used_rect()
r_x = used_rect.size.x * 16
r_y = used_rect.size.y * 16
center = used_rect.get_center()
var local_center = boundary_tilemap.map_to_local(center)
center = boundary_tilemap.to_global(local_center)
```

Then I get the formula for the ellipse

$$\frac{((X - C_x) \cos(\theta) + (Y - C_y) \sin(\theta))^2}{(R_x)^2} + \frac{((X - C_x) \sin(\theta) - (Y - C_y) \cos(\theta))^2}{(R_y)^2} = 1$$

There:

- (C_x, C_y) is the center of the Ellipse.
- R_x is the Major-Radius, and R_y is the Minor-Radius.
- θ is the angle of the Ellipse rotation.

Using the formula from Ephshtain's post (2018):

- The angle of rotation is the same angle of rotation of the tilemap
- The major radius is the rectangle's length, the minor radius is the rectangle's width
- The center of the ellipse is also the center of the rectangle
- If the total area is ≤ 1 , it is inside the ellipse, otherwise it is outside (Dan, 2011)

```
func is_part_of_ellipse(x,y) -> bool:
    var res = (((x-center.x)*cos(theta_ellipse) + (y-center.y)*sin(theta_ellipse))**2)/(r_x)**2
    res += (((x-center.x)*sin(theta_ellipse) + (y-center.y)*cos(theta_ellipse))**2)/(r_y)**2
    if res <= 1:
        return true
    return false
```

Not be behind a filled area of the tilemap

1. I got a list of all the occupied cells in the tilemap, converted it to global position and sorted it in ascending order (smallest to largest) in **_ready()**

```
var cells = floor_tilemap.get_used_cells()
    for c in cells:

floor_cells.append(floor_tilemap.to_global(floor_tilemap.map_to_local(c)))
    floor_cells.sort_custom(func(a, b):
        return a.x < b.x)
```

2. I had a function that would perform **binary search** to find the index of a matching x position, taken from GeeksForGeeks (2014)

- Binary search is essentially a function that iteratively splits up the sorted range of numbers in the middle. At every split, it checks:
 - Is the value in the middle of the array equal to the desired value of **x**
 - If yes, return the index containing the position of this value inside the array
 - If not, there are two cases:
 - Middle value is greater than **x** → ignore left half of the array
 - Middle value is less than **x** → ignore right half of array
- Binary search is done in **O(log n)** time instead of **O(n)** which for a relatively large map does offer some speedup

```
func return_x_binary_search(x) -> int:
  # binary search
  var low = 0
  var high = floor_cells.size() - 1
  while low <= high:
    @warning_ignore("integer_division")
    var mid = low + (high - low) / 2
    # Check if x is present at mid
    if floor_cells[mid].x == x:
      return mid
    # If x is greater, ignore left half
    elif floor_cells[mid].x < x:
      low = mid + 1
    # If x is smaller, ignore right half
    else:
      high = mid - 1

  # If we reach here, then the element was not present
  return -1
```

3. If this function returned an index, I had another function that would find all coordinate pairs that started with the same **x** (simply increase and decrease the index from the **x** that was known until the value of **x** changes)

```
func find_all_with_same_x(index) -> Array[int]:
  # returns list of indices with same x value
```

```

var results: Array[int] = []
var target_val = floor_cells[index].x
var i = index
while i >= 0 and floor_cells[i].x == target_val:
    results.append(floor_cells[i])
    i -= 1
i = index + 1
while i < floor_cells.size() and floor_cells[i].x == target_val:
    results.append(floor_cells[i])
    i += 1
return results

```

4. Finally, search the list of coordinates with a matching x manually until I got a matching y

```

func is_point_in_wall(x,y) -> bool:
    var index = return_x_binary_search(x)
    if index == -1:
        return false
    else:
        var indices = find_all_with_same_x(index)
        for i in indices:
            if floor_cells[i].y == y:
                return true
        return false

```

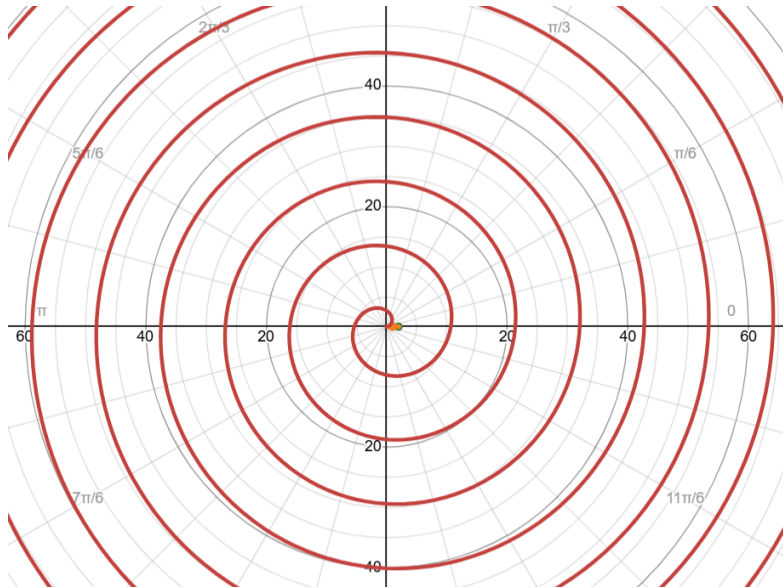
Perhaps I could optimize this for searching with y as well, but since the x values had a higher range than the y values, I felt like making the search for a matching x more efficient made more sense. Plus, since I will use a bounded spiral to determine where the shrimp can spawn, this will also limit the number of possible y coordinates a shrimp can spawn in

Lead the player towards the final area

This one is tied to the actual spawning function of a spiral which I will explain more below. But if I decrease theta everytime it hits an invalid point, there will be a new search area closer to the final area where the shrimp can spawn in that will 1. be different from the previous area and 2. lead the player closer to the final area

Spawning algorithm for the shrimp

The one I chose was a spiral.



I chose this because the shrimp would then traverse large portions of the map and also go slowly towards the center. The equation I used was $r = 1.7 * \theta$, which I got by just shifting values around until there was a gap of 7 units from the center because my bounding box for the final area was 7 tiles in height.

I had a second spiral that was shifted by **delta_a** (approximately half the size of the final area) These two created the spiral bounds of where the player could spawn.

```
func spiral_bounds(angle) -> Array[float]:  
    var lower_bound = angle*1.7  
    var upper_bound = delta_a + angle*1.7  
    return [lower_bound, upper_bound]  
  
func is_point_in_spiral(r, theta) -> bool:  
    var spiral_bounds = spiral_bounds(theta)  
    if spiral_bounds[0] <= r && r <= spiral_bounds[1]:  
        return true  
    return false
```

delta_a here is calculated in the **_ready()** function and is hard coded.

```
delta_a = final_area.size.x * 16 / 2
```

This probably should be abstracted out in order to make the code more reusable if there was another character that also needed to be spawned in the same way

Since the spiral goes counterclockwise, in order to start the shrimp towards the bottom, I needed to decrease theta by a fixed amount every time until it reached 0 at the center of the spiral. I chose a value of **20*PI** as a start where it would decrease by **PI/12** every time the loop went off.

```
while current_theta > 0:
    var bounds = spiral_bounds(current_theta)
    var r = randf_range(bounds[0], bounds[1])
    var pos = convert_to_cartesian(r, current_theta)
    if !is_part_of_ellipse(pos.x, pos.y):
        current_theta -= CHANGE_THETA
        continue
    else:
        if !is_point_in_spiral(r, current_theta) or
is_point_in_wall(pos.x, pos.y):
            continue
        elif final_area.has_point(pos):
            #spawn_shrimp(pos.x, pos.y)
            shrimp.global_position = pos
            break
        else:
            #spawn_shrimp(pos.x, pos.y)
            shrimp.global_position = pos
            current_theta -= CHANGE_THETA
            await get_tree().create_timer(TIMER_LENGTH).timeout
```

At every theta step:

- It calculates the upper and lower values of r that can be had for the given theta
- A random r value is chosen
- The coordinate pair (r, theta) is converted to cartesian coordinates (x,y) for checking
- If the coordinate is outside the ship, theta is decreased (move closer to the center)

- If the coordinate is outside the spiral bounds or if it is overlapping with a tile on the tilemap, move on
- If the coordinate is inside the final area, stop the respawning process
- Otherwise, spawn the shrimp, change the theta and wait for a predetermined time before trying to spawn the shrimp again

TIMER_LENGTH is set to 3 at this point. This is already extracted as a global variable that can be changed for longer/shorter pauses between spawns of the shrimp

There is the code **convert_to_cartesian** takes a polar coordinate (with r and theta) and converts it to x and y coordinates

```
func convert_to_cartesian(r, theta):
    var x = r * cos(theta)
    var y = r * sin(theta)

    # additionally, offset it so everything moves up
    y += final_area_center.y
    x += delta_a/2
    return Vector2(x,y)
```

However, since the spiral starts at (0,0), I wanted to move it up so the spiral would be centered around the final area.

This **final_area_center** is calculated in **_ready()**

```
final_area_center = final_area.get_center()
var local_center_final_area = final_area_tilemap.map_to_local(final_area_center)
final_area_center = final_area_tilemap.to_global(local_center_final_area)
```

Here is the code for spawning the shrimp

```
func spawn_shrimp(x,y) -> void:
    if current_shrimp:
        current_shrimp.queue_free()
    var _shrimp = shrimp.instantiate() as CharacterBody2D
    get_tree().current_scene.add_child(_shrimp)
    _shrimp.global_position = Vector2(x,y)
    _shrimp.add_to_group("exit")
```

```
mini_map.goal_object = _shrimp  
current_shrimp = _shrimp
```

- Checks if there already is a shrimp in the scene (stored in the variable **current_shrimp**)
- Instantiates the shrimp at the given location **(x,y)**
- Add the shrimp to the group of **exit** so the Game Manager knows that if the player were to enter in the **Area2D** surrounding the shrimp, it would load the next scene
- The minimap's **goal_object** is changed to this new shrimp so its position can be displayed
- The variable **current_shrimp** is assigned the reference to the **_shrimp** character

References

Epshtain, G. (2018, February 11). What is the parametric equation of a rotated Ellipse (given the angle of rotation) [Online Forum Post]. Mathematics Stack Exchange.

<https://math.stackexchange.com/questions/2645689/what-is-the-parametric-equation-of-a-rotated-ellipse-given-the-angle-of-rotatio>

Dan Is Fiddling By Firelight (2011, October 27). Check if a point is within an ellipse [Online forum post]. Mathematics Stack Exchange.

<https://math.stackexchange.com/questions/76457/check-if-a-point-is-within-an-ellipse>

GeeksforGeeks. (2014, January 28). Binary Search Algorithm Iterative and Recursive Implementation. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/binary-search/>

Mostly Mad Productions. (2025, September 15). Celeste-Style Wall Climbing & Stamina + Wall Jump | Godot 4.4 [Video]. YouTube.

<https://youtu.be/Pf1jNU4Da30?si=idDJl0Oxuta-vxXje>

DevDrache. (2024, October 9). How to Pick up and move Objects - Learn Godot 4 3D - no talking [Video]. YouTube. <https://youtu.be/UT11Zmx75yg?si=ESi7UhAbhRdcSjAb>

Gwizz. (2024, May 21). Godot 4 Push Objects Tutorial [Video]. YouTube.
<https://youtu.be/QIEhUKHHaEk?si=7cPD6wapOiH-LKx8>